

AT32M412 PMSM FOC Motor Control Library User Guide

Introduction

This document introduces how to use AT32M412-LV-MOTOR-EV evaluation board with AT32 PMSM FOC motor control library to tune various FOC control methods for driving a motor. These methods include Hall sensor, incremental encoder and magnetic encoder, and sensorless control.

Applicable products:

Part number	AT32M412/M416
-------------	---------------

Contents

1	Motor control library algorithms	5
2	Environment requirements	6
	2.1 Hardware requirements	6
	2.2 Software requirements	8
3	FOC motor control library documents and debugging.....	12
4	Special peripheral configurations on EVB	13
	4.1 U & V phase current sensing and bus current sensing	13
	4.2 Overcurrent protection.....	15
5	Revision history.....	16

List of tables

Table 1. Correspondence between Flash size and ROM.....	8
Table 2. Reference documents for demos with different rotor position estimation methods	12
Table 3 . Reference documents for demo debugging	12
Table 4 . Document revision history.....	16

List of figures

Figure 1. AT32M412-LV-Motor-EVB board connectors	7
Figure 2. AT32M412CBT7 ROM settings (KEIL).....	9
Figure 3. AT32M412CBT7 ROM settings (AT32IDE)	10
Figure 4. AT32M412CBT7 ROM settings (IAR).....	11
Figure 5. Document structure	13
Figure 6. Standalone mode circuit.....	14
Figure 7. Bus overcurrent detection block diagram.....	15
Figure 8. Phase current sensing feedback	15

1 Motor control library algorithms

This document contains the following motor control algorithms:

- Target motor type: Three-phase permanent magnet synchronous motor (PMSM)
- Control mode
 - FOC vector control
- Three-phase PWM method
 - SVPWM
- Phase current sensing methods
 - 2-shunt current sensing
 - 1-shunt current sensing and reconstruction method
- Rotor position estimation
 - Hall-effect position sensor
 - Incremental encoder
 - Magnetic (absolute) encoder
 - Sensorless method
- Sensor-based FOC sinusoidal control mode
 - Voltage vector control
 - Torque control (current vector control)
 - Speed control
 - Field weakening control
 - Positioning control
- Sensorless FOC sinusoidal control mode
 - Torque control (current vector control)
 - Speed control
 - Field weakening control
 - Upwind/Downwind startup
- Auto tuning functions
 - Hall state sequence self-learning (for Hall sensor only)
 - Magnetic encoder deviation angle self-calibration
 - Motor winding parameters identification
 - Current PI controller parameter self-tuning

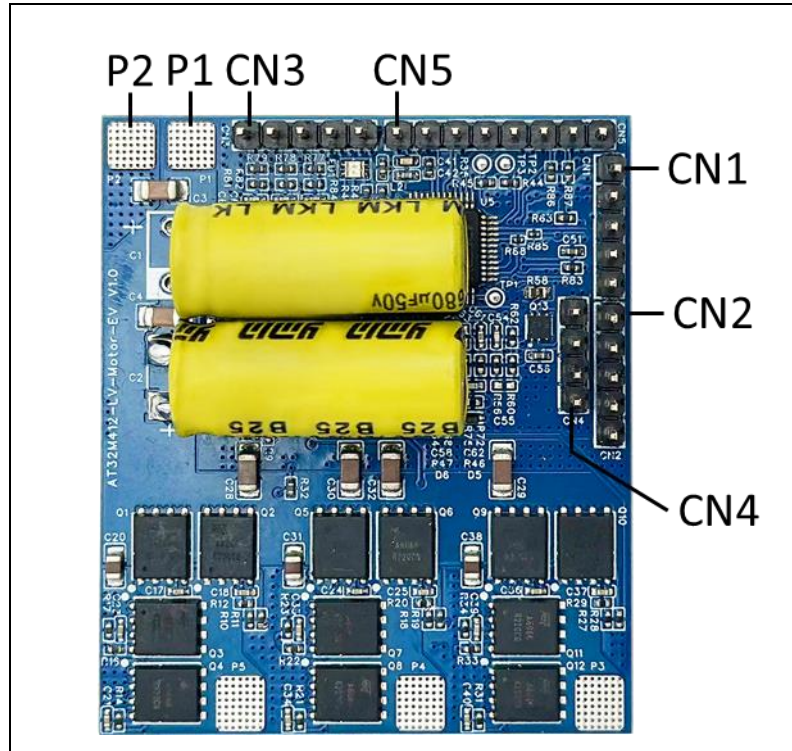
2 Environment requirements

2.1 Hardware requirements

Hardware resources required are listed below. For details about hardware configurations and wiring of AT32M412-LV-Motor-EV evaluation board, please refer to *UM0016_AT32_LV_Motor_EVB_V1_User_Manual*.

- Hardware resources
 - Windows®-based PC (Windows 8, Windows 10, Windows 11) to install motor UI tool and programming software
 - Artery AT-Link or 3rd-party programmer
 - One USB-to-TLL cable with a transmission rate of above 1.5 Mbps (required when a 3rd-party programmer is used) to connect the development board UART interface to PC for communication
 - AT32M412-LV-Motor-EVB motor development board
 - A three-phase PMSM (BLDC) AC motor
 - DC power supply
- Motor development board
 - Input voltage: 6V~45V
 - Maximum output phase current: 45APEAK
 - Overcurrent threshold point: 55APEAK
- Connection settings
 - 1) When a position sensor is used, connect Hall sensor (incremental encoder) to the EVB board CN3.
 - 2) When an external control input is used, connect the analog signal (0~3.3V) to CN2 PIN2, or pulse width modulation input (PWM IN) signal to CN2 PIN3.
 - 3) Connect AT-Link programmer (or 3rd-party programmer) to EVB board CN1.
 - 4) Connect AT-Link programmer (or USB-to-TLL cable) TX/RX cable to CN5 PIN3/PIN4.
 - 5) Program Artery AT32M412 motor control demo project.
 - 6) Remove the VDD power wire of the programmer on CN1.
 - 7) Connect the three-phase motor U/V/W to EVB P3/P4/P5, respectively.
 - 8) Adjust the DC supply voltage/current.
 - 9) Connect power supply (Li-polymer) positive to EVB P1 and negative to EVB P2, and then power on.
 - 10) Operate Artery motor UI tool (ArteryMotorMonitor), set parameters and control the motor to run.

Figure 1. AT32M412-LV-Motor-EVB board connectors



2.2 Software requirements

- 1) Take the Hall FOC control method as an example. Open the demo project at AT32M412_MC_Library_Project\at32m412_lv_motor_ev\at32m412\pmsm_foc_hall_sensor (note [1]).
- 2) Run ArteryMotorMonitor.exe directly (installation not required).
- 3) In Keil IDE, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU, as shown in Table 1. For example, for AT32F413RCT7 with a 256 KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 3). In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4). In IAR Systems modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5) (note [2]).

Table 1. Correspondence between Flash size and ROM

Flash size	128k	64k
IROM1 (address)	0x8000000	0x8000000
IROM1 (size)	0x1FC00	0x0FC00
IROM2 (address)	0x801FC00	0x800FC00
IROM2 (size)	0x400	0x400

Note [1]: This demo project does not support Keil compiler version 5 and O0 optimization level for compiling simultaneously. Please use Keil compiler version 6 or O1 optimization level for compiling. In case of Keil_v5.33, since AT32 BSP source code does not support V6.15 compiler, the Keil compiler version 5 and O1 optimization level should be used.

Note [2]: The magnetic encoder demo project reserves a Flash memory for magnetic encoder deviation angle calibration lookup table; therefore, the correspondence between Flash size and ROM is different from that of other demo projects.

ROM settings in Keil, AT32IDE and IAR are shown below.

Figure 2. AT32M412CBT7 ROM settings (KEIL)

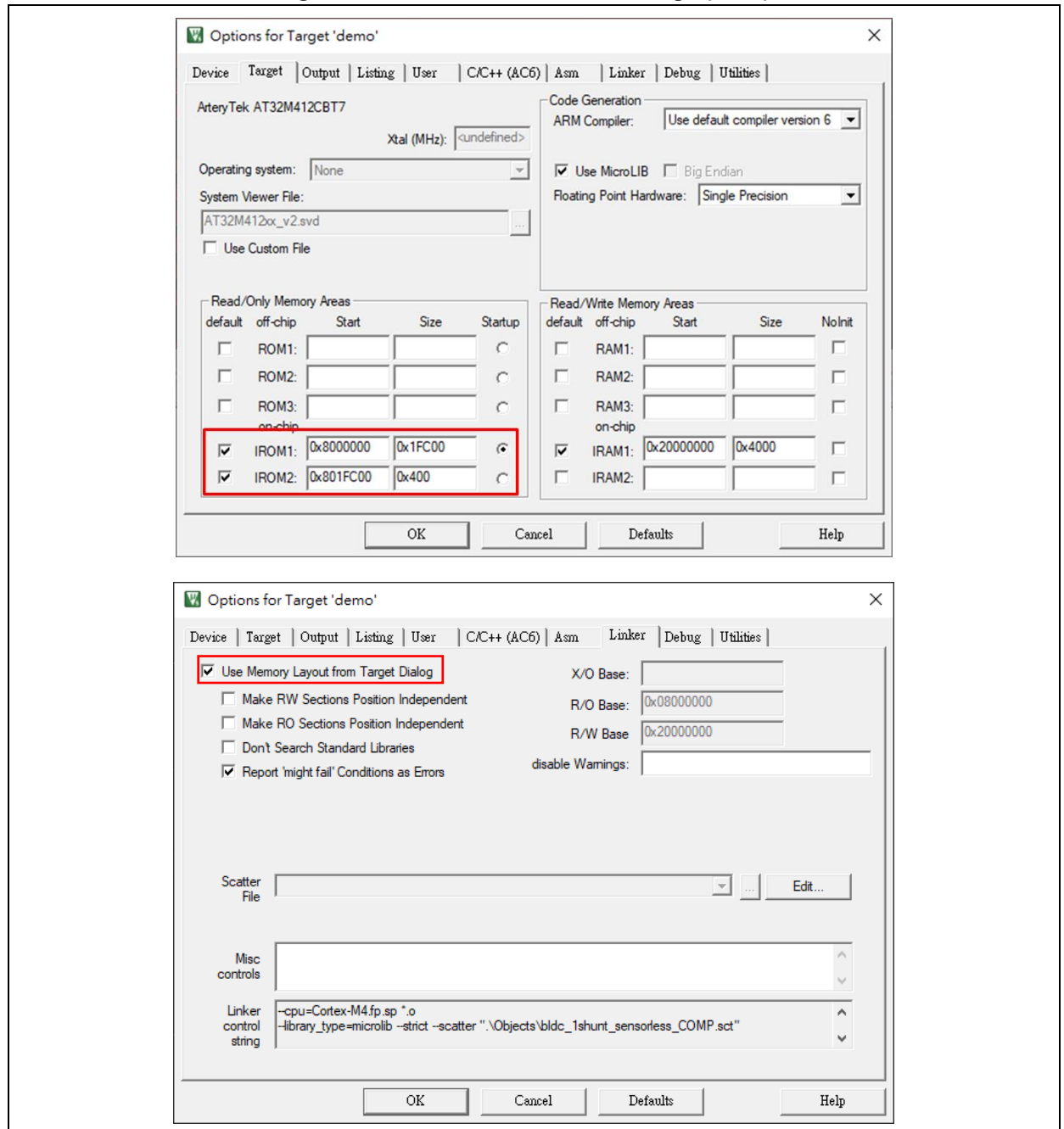


Figure 3. AT32M412CBT7 ROM settings (AT32IDE)

```

16  ** Environment : Arm gcc toolchain
17  **
18  ****
19  */
20
21  /* Entry Point */
22  ENTRY(Reset_Handler)
23
24  /* Highest address of the user mode stack */
25  _estack = 0x20004000; /* end of RAM */
26
27  /* Generate a link error if heap and stack don't fit into RAM */
28  _Min_Heap_Size = 0x200; /* required amount of heap */
29  _Min_Stack_Size = 0x400; /* required amount of stack */
30
31  /* Specify the memory areas */
32  MEMORY
33  {
34    FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 128K
35    MC_DATA (r)      : ORIGIN = 0x0801FC00, LENGTH = 0x400
36    RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 16K
37  }
38
39  /* Define output sections */
40  SECTIONS
41  {
42    /* The startup code goes first into FLASH */
43    .isr_vector :
44    {
45      . = ALIGN(4);
46      KEEP(*(.isr_vector)) /* Startup code */
47      . = ALIGN(4);
48    } >FLASH
49
50    .mc_data :
51    {
52      . = ALIGN(4);
53      . = ORIGIN(MC_DATA);
54      _MC_VectStoreAddr1 = .;
55      *(.MC_VectStoreAddr1);
56      . = ALIGN(4);
57      . = ORIGIN(MC_DATA) + 800;
58      _MC_VectStoreAddr2 = .;
59      *(.MC_VectStoreAddr2);
60      . = ALIGN(4);
61    } >MC_DATA

```

Figure 4. AT32M412CBT7 ROM settings (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0801FC00 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0801FC00;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0801FC00 + 0x400 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20003FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /**** End of ICF editor section. ****ICF****/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 FOC motor control library documents and debugging

FOC motor control library documents are detailed below:

- Hardware circuits

For details about AT32M412 low-voltage general-purpose development board hardware circuits, please refer to *UM0016_AT32M412_LV_Motor_EVB_V1_User_Manual*.

- Documents for different control methods

The AT32M412 low-voltage general-purpose development board is designed by Artery for AT32M412 series. All motor control evaluation boards use the same motor control library, and complete documents are provided based on different rotor position estimation methods. These documents include the file structure, macro definitions and state machine description.

Table 2. Reference documents for demos with different rotor position estimation methods

Rotor position estimation	Reference document
Hall sensor	AN0167_AT32_PMSM_FOC_Hall_Demo
Hall sensor (E-bike)	AN0194_AT32_PMSM_FOC_Hall_Demo(E-Bike-Scooter)
Incremental encoder	AN0166_AT32_PMSM_FOC_Incremental_Encoder_Demo
Magnetic encoder	AN0195_AT32_PMSM_FOC_Magnetic_Encoder_Demo
Sensorless	AN0168_AT32_PMSM_FOC_Sensorless_Demo

- General functions and macro definitions

For details about general functions and macro definitions, please refer to *AN0064_AT32_Motor_Control_Library_User_Guide*.

- Tuning procedure

The below table list the quick start guides for demos with different rotor position sensing methods and estimation technique.

Table 3 . Reference documents for demo debugging

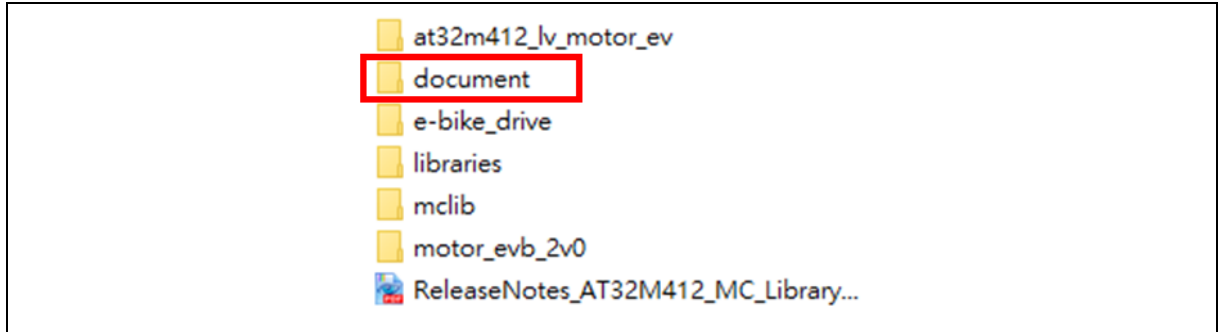
Rotor position estimation	Reference document
Hall sensor	AN0218_AT32_PMSM_FOC_Hall_Quick_Start_Guide
Hall sensor (E-bike)	AN0219_AT32_PMSM_FOC_Hall(E-Bike-Scooter)_Quick_Start_Guide
Incremental encoder	AN0217_AT32_PMSM_FOC_Incremental_Encoder_Quick_Start_Guide
Magnetic encoder	AN0196_AT32_PMSM_FOC_Magnetic_Encoder_Quick_Start_Guide
Sensorless	AN0220_AT32_PMSM_FOC_Sensorless_Quick_Start_Guide

- Artery motor UI tool

- Refer to *AN0063_AT32_Motor_Monitor_Application_Note* for the use of Artery motor UI tool.
- Refer to *AN0199_AT32_Motor_Monitor_Protocol* for software transmission protocol of Artery motor UI tool.

These documents are available on Artery's official website. Documents including AN0064, AN0166, AN0167, AN0168, AN0194, AN0195, AN0196, AN0217, AN0218, AN0219, AN0200 are packed in "document" folder of motor control library, as shown below. Besides, AN0063 and AN0199 are also packed in the ArteryMotorMonitor.exe.

Figure 5. Document structure



4 Special peripheral configurations on EVB

The AT32M412 low-voltage general-purpose evaluation board uses the built-in operational amplifier (OP) together with ADC for U & V phase current sensing and bus current sensing, and uses the internal comparator (CMP) together with DAC for bus overcurrent protection (OCP). This section introduces how the OP works in U & V phase current sensing and bus current sensing and how to implement bus OCP.

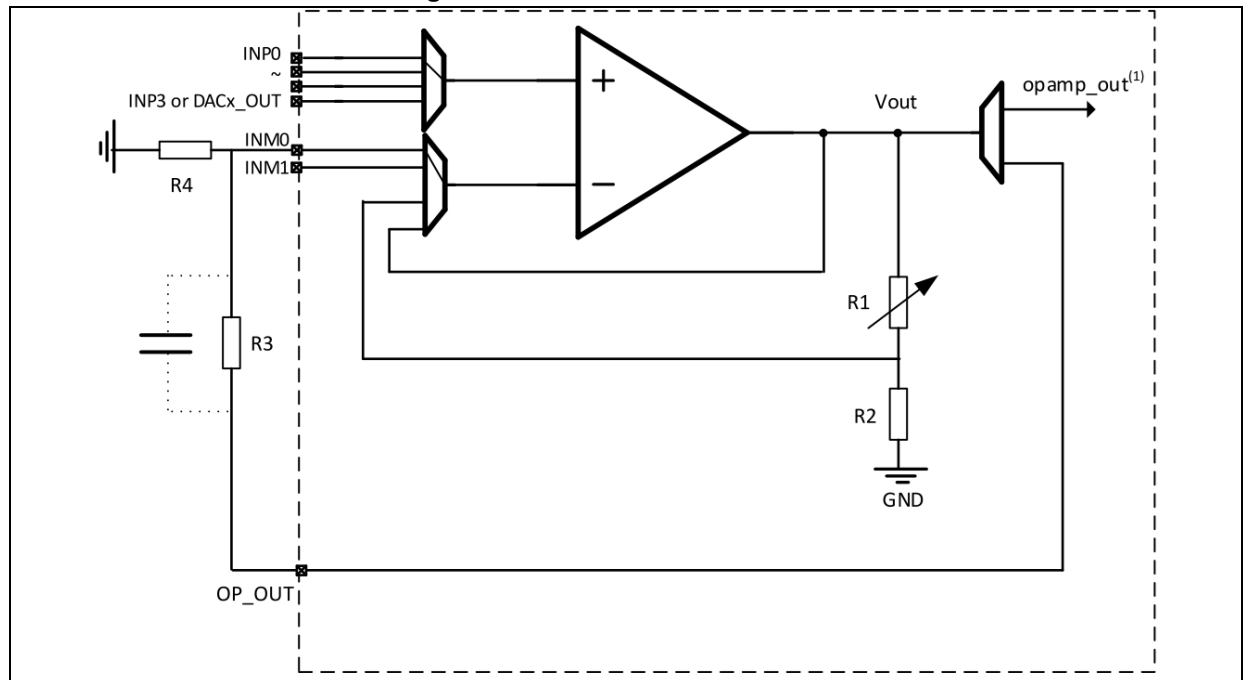
4.1 U & V phase current sensing and bus current sensing

The motor control EVB utilizes three-phase full-bridge low-side MOSFET conduction voltage drop to estimate motor current. Since the conduction voltage drop is small, the AT32M412 built-in OP1 and OP4 are used to amplify V and U phase signals and then read them via ADC CH2 and CH12. For details, refer to *UM0016_AT32M412_LV_Motor_EVB_V1_User_Manual*. To amplify MOSFET conduction voltage drop, a large current will flow through the resistor at the non-inverting terminal, which means that the OP Standalone mode should be selected, instead of PGA mode. In addition, the EVB board is connected in series with a 2 mΩ current sensing resistor at the bus ground terminal, and uses the MCU built-in OP to amplify signals. The OP standalone mode (its amplification ratio can be adjusted by external resistance) is selected to obtain an appropriate current sensing range. Follow the below steps to enable OP Standalone mode:

- Set OPx_CTRL.INMSEL[1:0] = 0x.
- Set the corresponding INP/INM channel as analog mode.
- OP enters high-speed mode if OPx_CTRL.OPFSTMD = 1.
- OP enters Standalone mode by setting OPx_CTRL.OPEN = 1.

The OP standalone mode circuit is shown below. Its open loop amplification ratio is dependent on external resistors R3/R4.

Figure 6. Standalone mode circuit



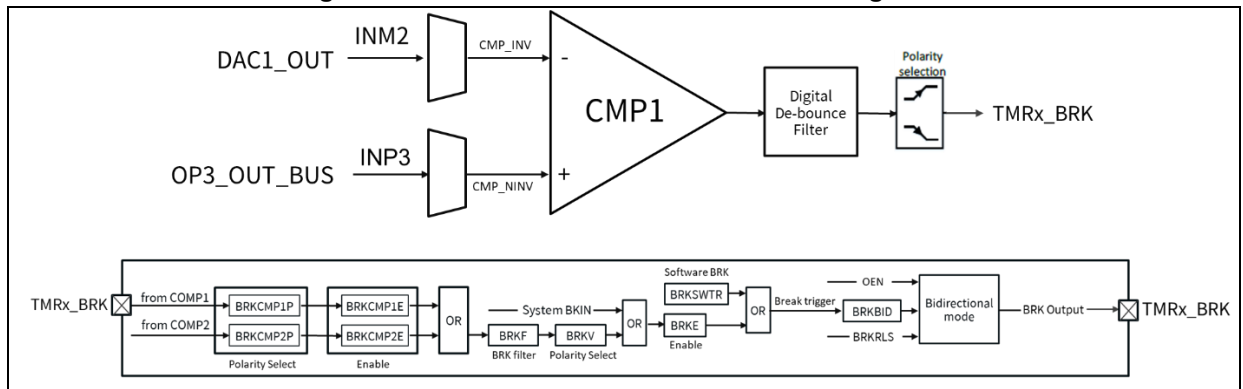
The OP output (OP_OUT) pin is also used as the input of ADC, which can be used for ADC sampling to feed back of motor current signals in a real-time manner.

4.2 Overcurrent protection

The OP3 output pin PB1 is also used as ADC input ADC_CH9. It also internally connects to CMP1 INP3 and to CMP1 INM2 in combination with DAC1 output. The bus overcurrent threshold is determined by DAC output value. The CMP output is connected to TMR1 brake input.

- Set CMP1 high speed by setting CMPSSSEL=0.
- Set CMPHYST = 0.
- Set timer brake input polarity active low by setting BRKCMP1P=1.
- Enable brake input by setting BRKCMP1EN=1.
- Set the number of brake filtering = 3 (BRKF=3)
- Set brake input active low by setting BRKV=0.
- Enable TMR1 brake by setting BRKEN=1.

Figure 7. Bus overcurrent detection block diagram



The overcurrent protection threshold is set through the macro definition OCP_CURRENT in *motor_control_drive_param.h* file. The compiler calculates the DAC output voltage value according to the macro definitions.

Figure 8. Phase current sensing feedback

```
#define DAC_VREF_SOURCE      DAC_VDDA
#define OCP_CURRENT          (55.0)
#define BUS_CURR_CMP_OCP_VOLT ((OCP_CURRENT*RDC_SHUNT*DC_OP_GAIN)+IDC_OFFSET_VOLT)
#define TMR_BRK_FILTER_COUNT (3)
```

5 Revision history

Table 4 . Document revision history

Date	Version	Revision note
2025.08.28	1.0.0	Initial version
2025.10.28	1.0.1	Added support for AT32M416 model and related magnetic encoder project description.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.